

Source3: A Rule-Based Pipeline for Equation Annotation in Scientific Papers

1 Overview

1.1 Problem Analysis

The goal of Source3 is to produce structured annotations for a curated set of mathematical equations drawn from 46 arXiv papers in the domain of quantum computing and quantum information. For each reviewed equation the system must extract three types of information:

1. a *meaning* — a natural-language sentence that describes what the equation expresses;
2. *symbol definitions* — a mapping from every mathematical symbol in the equation to its natural-language definition; and
3. *inter-equation relations* — a classification of how the equation relates to every other reviewed equation in the same paper (e.g. derived from, equivalent, or unrelated).

A central constraint is that all output strings must be verbatim spans from the source paper; no text may be generated or paraphrased. This requirement makes the outputs fully traceable and avoids hallucination, but it forces every extraction step to locate the correct span within the document rather than synthesising an answer.

Three technical challenges characterise the problem. First, *equation alignment*: the human-curated equation list identifies each equation by its LaTeX source and, where available, by a visible label or a DOM anchor extracted during a prior annotation pass. The HTML rendered by arXiv uses a different identifier scheme and may contain minor formatting differences, so a multi-strategy matching procedure is required to link each reviewed equation to its unique DOM node unambiguously. Second, *meaning localisation*: the sentence that best describes an equation is not always the one immediately preceding it. It may occur in a cross-reference later in the paper, in an equation caption, or may be signalled by a cue phrase such as “is defined as” or “Eq. (N) gives”. Naïve proximity heuristics are insufficient; ranking is needed. Third, *symbol disambiguation*: the same symbol (e.g. ψ) may appear in many equations with distinct roles across a paper. The correct definition must be retrieved for each specific occurrence without confusion from other uses.

1.2 Approach Overview

Source3 is a fully deterministic, non-generative pipeline consisting of ten sequential stages. It uses a semantic embedding model (MathBERT [1], specifically the `witiko/mathberta` checkpoint) for dense retrieval, BM25 for lexical matching, and hand-crafted linguistic patterns for high-precision span extraction. No large language model is invoked for generation.

The pipeline is divided into four logical phases.

Ingestion phase (Stages 1–4). Raw HTML is downloaded from arXiv and cached locally (Stage 1). Each HTML document is then parsed into a hierarchical model comprising sections, paragraphs, sentences with stable identifiers, and display equation blocks with their associated MathML and original $\text{\LaTeX}$ source (Stage 2). The document is subsequently segmented into typed retrieval chunks — individual sentences, paragraphs, raw equation neighbourhoods (up to five sentences on each side of an equation), and cross-reference spans (Stage 3). Finally, each reviewed equation is aligned to its DOM node using a cascade of six matching strategies ordered by specificity, from exact DOM-ID match down to normalised- $\text{\LaTeX}$ overlap; ambiguous matches are rejected (Stage 4).

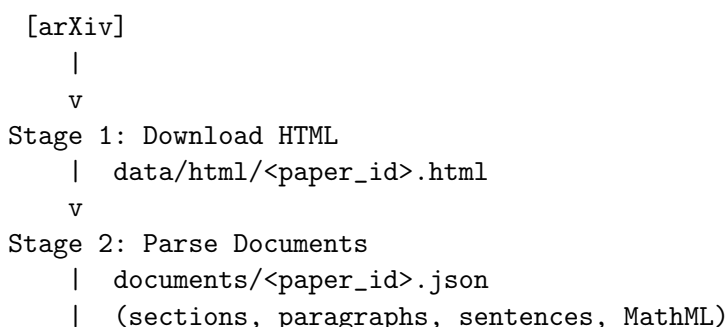
Embedding phase (Stage 5). A single MathBERT instance is loaded once per paper. Two families of vectors are computed and stored in a per-paper `.npz` file. Family A is equation-centric: for each reviewed equation, up to twelve vectors are produced encoding the equation alone, a summary context (equation with five surrounding sentences), and individual sentence-conditioned views (equation paired with each neighbouring sentence). Family B is sentence-centric: every sentence and neighbourhood chunk in the document is embedded to support symbol definition retrieval later. All vectors are mean-pooled over the final hidden state, L2-normalised, and stored as `float32[768]`.

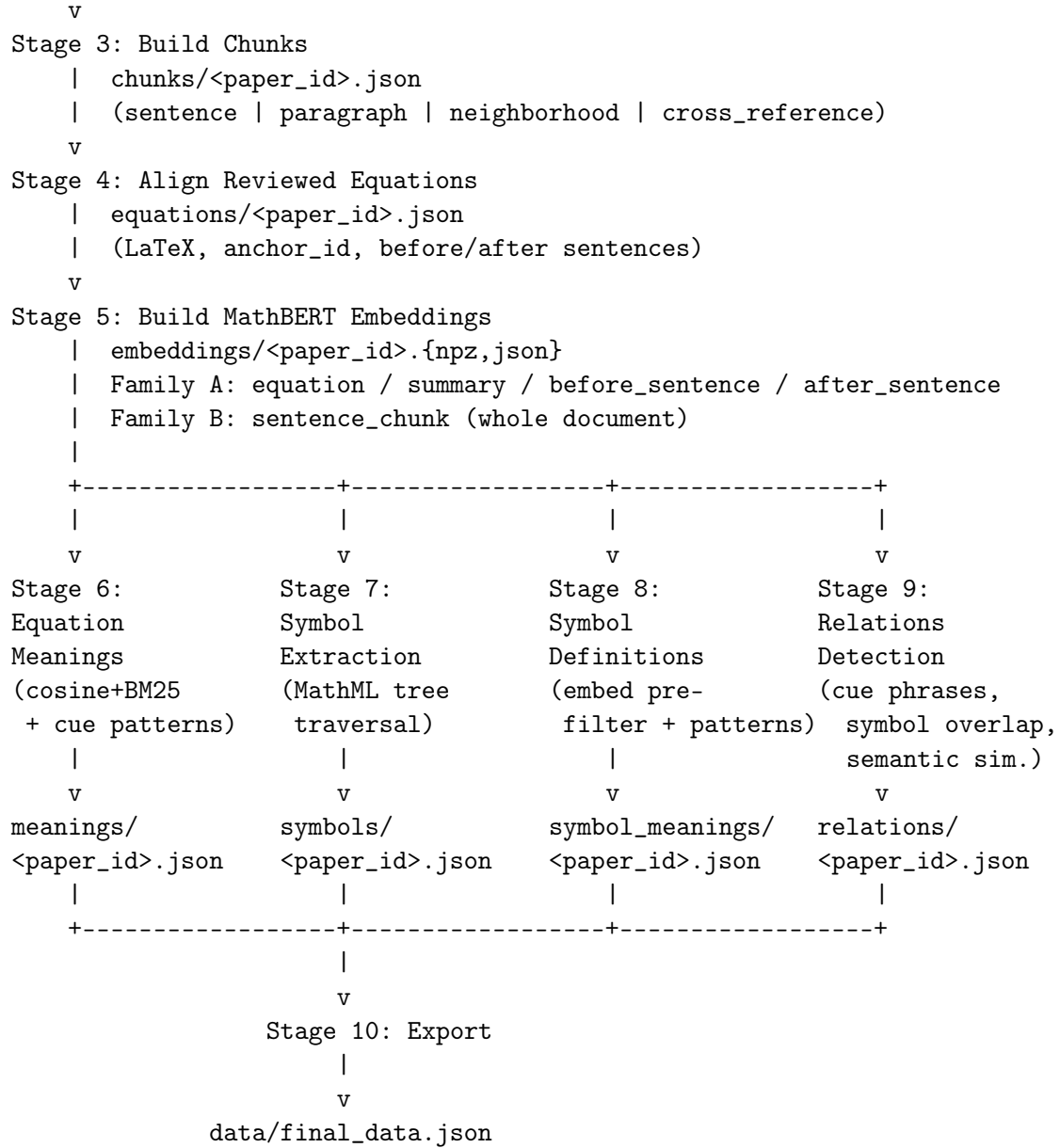
Extraction phase (Stages 6–9). Equation meanings are retrieved by first collecting candidate sentences from three sources in priority order — cross-references, cue-pattern matches, and proximity fallback — then reranking survivors with a weighted score combining cosine similarity to the equation’s summary vector, BM25, source-type bonus, and proximity (Stage 6). A post-processing pass refines or discards low-quality candidates (Stage 6b). Symbols are extracted from the MathML tree of each equation, with composite identifiers (subscripted, superscripted, decorated) resolved before flat leaf collection, and a $\text{\LaTeX}$ -tokeniser fallback applied when MathML is absent (Stage 7). Symbol definitions are located in two steps: an embedding pre-filter computes cosine similarity between a query formed from the symbol’s aliases and all sentence-chunk vectors, retaining the top 30 candidates; ordered linguistic patterns (“where *sym* is”, “*sym* denotes”, “let *sym* be”, etc.) are then applied to those candidates only (Stage 8). Inter-equation relations are scored using explicit citation cues, derivation and equivalence cue phrases, Jaccard overlap of canonical symbol sets, section proximity, and cosine similarity between summary vectors; relations are graded as *strong*, *potential*, or *none* (Stage 9).

Export phase (Stage 10). The four intermediate artefacts (equations, meanings, symbol definitions, relations) are joined on equation identifier, validated against a checklist of schema invariants, and written atomically to `data/final_data.json` via a temporary file and rename.

1.3 Pipeline Diagram

The diagram below shows the data flow through the ten stages.





References

- [1] S. Peng, K. Yuan, L. Gao, and Z. Tang, "MathBERT: A Pre-trained Model for Mathematical Formula Understanding," *arXiv preprint arXiv:2105.00377*, 2021.